Transferência de arquivos P2P

Sistemas Distribuídos - TP2 | CEFET-MG | 2026/2
Professora: Michelle Hanne. Preparado para Augusto Coelho Fagundes.
Implementação, testes e redação: Codex, assistente de IA da OpenAI, a pedido do usuário. Data: 29/10/2026.

Código-fonte: https://github.com/Augusto42/sd-tp2-p2p-2026
Repositório privado: o titular precisa conceder acesso ao avaliador. O ZIP contém uma cópia independente do código e das evidências.

1. Arquitetura e decisões

Cada peer é um processo Python com um servidor TCP e tarefas clientes no mesmo event loop asyncio. Ele atende solicitações enquanto baixa de seus vizinhos. A biblioteca padrão reduz dependências e permite concentrar o trabalho em protocolo, concorrência e integridade.

Os vizinhos são estáticos, configurados por IP e porta. Não foi usado tracker, pois é opcional. A matriz de testes usa A -> B -> C -> D para forçar o repasse por peers intermediários. Um teste adicional utiliza malha completa.

O arquivo é dividido logicamente em blocos de 1.024 ou 4.096 bytes. O metadado JSON informa versão, tamanho, tamanho de bloco, nome informativo, hashes por bloco e hash completo. Todos recebem esse metadado previamente; ele é considerado confiável.

Os blocos ficam em um arquivo parcial em disco, gravados por offset, evitando carregar todo o arquivo em memória. O conjunto have registra os blocos verificados; busy reserva pedidos em voo e evita que duas tarefas solicitem o mesmo bloco ao mesmo tempo.

Após cada bloco passar na checagem de tamanho e SHA-256, ele fica disponível para upload. Ao concluir, verifica-se o SHA-256 completo e renomeia-se o parcial para arquivo_final.bin. O peer continua servindo até receber SIGINT ou SIGTERM.

2. Protocolo e implementação

TCP fornece um fluxo de bytes, não mensagens. Por isso cada frame usa prefixo de comprimento de 4 bytes big-endian e corpo de até 8 MiB. readexactly trata frames divididos em múltiplas leituras. Conexões persistentes evitam reconectar a cada bloco.

Mensagem	Conteúdo / função
have (JSON)	Pedir lista dos índices disponíveis
get (JSON)	Pedir índice de bloco e identificar arquivo pelo hash
J + JSON	Responder inventário de blocos
B + bytes	Entregar bloco binário sem base64
E + texto	Indicar arquivo incorreto ou bloco indisponível

Um pedido fica em voo por conexão, dispensando identificador de correlação. Cada vizinho possui sua própria tarefa de download. O servidor mantém uma tarefa por conexão recebida. Gravação/atualização do conjunto e leitura do bloco não contêm await; assim, outras tarefas não alteram o ponteiro compartilhado entre seek e read/write.

Conexão tem timeout de 2 s; resposta, 5 s; servidor ocioso, 15 s. Falhas liberam reservas e tentam novamente após 100 ms; falta de blocos novos gera nova consulta após 20 ms. Estes valores privilegiam uma demonstração em rede local. Não há limite global de espera.

Hashes por bloco rejeitam dados corrompidos antes de publicá-los. O hash final confirma remontagem. O nome do metadado é informativo: a saída usa nome local fixo, evitando usar dados remotos como caminho de escrita.

3. Procedimento experimental

Executaram-se 24 combinações: 2/4 peers x 1.024/4.096 bytes x 10/20 KiB, 1/5/10/20 MiB. KB e MB do enunciado foram interpretados como unidades binárias. Cada caso usa conteúdo aleatório, processos independentes e sockets TCP reais em loopback; nenhum resultado é simulado.

Cada transferência verifica tamanho, SHA-256 e origem dos blocos nos logs. Os testes preservam metadados, eventos JSONL, verificações e tabelas CSV/JSON; os binários temporários são descartados. A repetição gera novos dados. Cada configuração teve uma execução; não há intervalos de confiança.

4. Resultados da matriz completa

Arquivo	2 peers / 1 KiB	2 peers / 4 KiB	4 peers / 1 KiB	4 peers / 4 KiB
10 KiB	0.3279 s	0.3377 s	0.5871 s	0.5034 s
20 KiB	0.2655 s	0.3511 s	0.6328 s	0.5799 s
1 MiB	0.5519 s	0.3902 s	0.8456 s	0.6416 s
5 MiB	1.0568 s	0.5065 s	2.0080 s	0.7839 s
10 MiB	1.3361 s	0.6278 s	2.5660 s	1.0588 s
20 MiB	2.5323 s	0.9107 s	4.6279 s	3.4303 s

Integridade: 24/24 combinações aprovadas. Todos os arquivos finais tiveram tamanho e SHA-256 iguais ao original. Os logs dos casos em cadeia confirmam que B recebeu de A, C de B e D de C.

Em 20 MiB, dois peers levaram 2,5323 s com blocos de 1 KiB e 0,9107 s com 4 KiB; quatro peers levaram 4,6279 s e 3,4303 s, respectivamente. Nessa execução, blocos maiores reduziram a quantidade de requisições e eventos de log, e apresentaram menor tempo.

Quatro peers na cadeia precisam entregar três cópias e atravessar peers intermediários; não é uma comparação de aceleração de um único download. Os tempos incluem inicialização, cópia da semente, polling e logs, e não apenas transmissão de bytes. Arquivos pequenos são especialmente influenciados por esses custos.

O campo aggregate_mib_s do CSV é tamanho x (peers - 1) / tempo / 2^20: taxa agregada de cópias entregues, não capacidade física de enlace. Rede local em loopback, caches e escalonamento limitam qualquer generalização para LAN/WAN.

Ambiente detalhado em resultados/ambiente.json. Logs por caso em resultados/<caso>/peer_N.jsonl; verificacao.json associa hashes, fontes, tempo e contagem de repasses parciais.

5. Casos adicionais e avaliação

Compartilhamento parcial: quatro processos, 131.123 bytes, blocos de 1 KiB e atraso artificial de 1 ms por resposta. Conclusão em 1,4118 s; 216 envios ocorreram por leechers antes da conclusão de seus downloads. O atraso facilita observar a concorrência e não integra a matriz de desempenho.

Seeder tardio: o seeder inicia 700 ms depois do leecher. Houve sete tentativas de reconexão e conclusão correta em 1,0287 s. O caso demonstra recuperação de indisponibilidade inicial; não simula reinício no meio da transferência.

Malha completa: quatro peers, 1 MiB e blocos de 4 KiB. Conclusão correta em 1,1444 s; foram observadas múltiplas fontes em dois leechers e 30 envios antes da conclusão. A seleção depende do escalonamento, sem justiça garantida entre vizinhos.

Testes unitários: 10/10 aprovados. Cobrem último bloco parcial e montagem fora de ordem, corrupção, tamanho incorreto, arquivo vazio, metadado inválido, tamanho de bloco inválido, hash final divergente, frame fragmentado, frame truncado e limite de frame.

6. Limitações e conclusão

A solução atende à simetria, fragmentação, registro local de blocos, redistribuição durante o download e montagem verificada. Usa operações de disco síncronas curtas no event loop; a aplicação não garante durabilidade contra queda de energia. Não implementa autenticação/TLS, rarest-first, descoberta de peers, retomada persistente nem distribuição automática de metadados. Vizinhos são cooperativos e o metadado é confiável.

As 27 integrações e os 10 testes unitários passaram. Isso comprova os cenários executados, sem demonstrar comportamento em toda falha possível. Melhorias futuras: retomada com bitmap persistente, inventários incrementais, agendamento de blocos e ensaios repetidos em máquinas/redes distintas.

Referências: enunciado SD_TP2_2_2026.pdf, seções 1 a 3, pp. 1-2; código e evidências no repositório acima. Instruções reproduzíveis no README e especificação detalhada em docs/protocolo.md.